

Hotel Booking System Database Documentation

Overview

This system is designed to manage hotels, rooms, users, bookings, and payments with a focus on **data integrity, concurrency control, and auditing**. It leverages **InnoDB** features in MySQL such as **foreign keys, transactions, row-level locking**, and **JSON columns** for flexible data storage.

Tables and Relationships

1. Users

- Stores system users including customers and administrators.
- Fields:
 - `id`: Primary key.
 - `name`, `email`, `phone`: Personal information.
 - `password_hash`: Secure password storage.
 - `role`: User type (USER or ADMIN).
 - `created_at`, `updated_at`: Timestamps.
- Constraints:
 - Unique email.
- Usage:
 - Linked to bookings and transaction logs.

2. Hotels

- Stores hotel information.
- Fields:
 - `id`, `name`, `address`, `city`, `state`, `country`, `zip_code`.

- `rating`: Average hotel rating.
- `amenities`: JSON array to store dynamic features.
- `description`: Optional description of the hotel.
- Usage:
 - One-to-many relationship with `rooms`.

3. Room Types

- Defines room characteristics for multiple hotels.
- Fields:
 - `id`, `name`, `description`, `capacity`, `base_price`, `amenities`, `size_sqft`, `bed_type`.
- Usage:
 - Linked to `rooms` table to define the type of each room.

4. Rooms

- Represents individual rooms in a hotel.
- Fields:
 - `id`, `hotel_id`, `room_type_id`, `room_number`, `floor`, `status`.
- Constraints:
 - Unique room number per hotel.
 - Foreign key to `hotels` (CASCADE on delete).
 - Foreign key to `room_types` (RESTRICT on delete).
- Indices:
 - Room status for fast availability searches.
- Usage:
 - Linked to `bookings`.

5. Bookings

- Records customer reservations.
- Fields:

- `id, user_id, room_id, checkin_date, checkout_date, total_price, status, guest_count, special_requests.`
- Constraints:
 - Check constraint ensures `checkout_date > checkin_date`.
 - Foreign keys to `users` (CASCADE) and `rooms` (RESTRICT).
- Indices:
 - Optimized for room availability and user booking history.
- Usage:
 - Central table for booking management.
- Advanced Techniques:
 - Row-level locks (`SELECT ... FOR UPDATE`) during booking creation to prevent double-booking.

6. Payments

- Handles payment transactions for bookings.
- Fields:
 - `id, booking_id, amount, currency, method, status, transaction_ref, payment_details.`
- Constraints:
 - Foreign key to `bookings` (CASCADE).
 - Unique transaction reference.
- Indices:
 - Optimized for booking-based queries.
- Usage:
 - Integrates financial transactions with bookings.
- Advanced Techniques:
 - Supports multiple payment methods with JSON storage for dynamic details.

7. Transaction Log

- Auditing table for all critical actions.
 - Fields:
 - `id, booking_id, user_id, action, table_name, record_id, old_values, new_values, details, ip_address, user_agent.`
 - Usage:
 - Tracks changes for compliance and debugging.
 - Advanced Techniques:
 - Stores before/after values in JSON.
 - Foreign keys to bookings and users.
-

Table Relationships

- **Users → Bookings:** One-to-many (`users.id → bookings.user_id`)
- **Hotels → Rooms:** One-to-many (`hotels.id → rooms.hotel_id`)
- **Room Types → Rooms:** One-to-many (`room_types.id → rooms.room_type_id`)
- **Rooms → Bookings:** One-to-many (`rooms.id → bookings.room_id`)
- **Bookings → Payments:** One-to-one (`bookings.id → payments.booking_id`)
- **Bookings / Users → Transaction Log:** One-to-many (`bookings.id → transaction_log.booking_id, users.id → transaction_log.user_id`)

This structure ensures **referential integrity** while enabling cascading deletions and updates where appropriate.

Advanced Database Techniques

1. Transactional Integrity

- Bookings are created inside **transactions** to ensure **atomicity**.
- If any step fails, the transaction rolls back.
- Guarantees that **room availability**, **payment**, and **transaction logs** are synchronized.

2. Row-Level Locking

- `SELECT ... FOR UPDATE` locks specific room rows during booking.
- Prevents **race conditions** where two users book the same room simultaneously.

3. Overlapping Booking Checks

- SQL checks overlapping dates to prevent double-booking.
- Uses a condition:
- `NOT (checkout_date <= checkin_date OR checkin_date >= checkout_date)`

4. Auditing and Logging

- Every booking action is logged with old/new values and metadata (IP, user agent).
- Supports **tracing**, **rollback**, and **compliance reporting**.

5. JSON Columns

- Flexible storage for `amenities` and `payment_details`.
- Supports **dynamic attributes** without altering schema.
- Indexing possible for efficient querying using **generated columns** if needed.

6. Data Consistency with Foreign Keys

- Cascading deletes for dependent data (users → bookings → payments).
 - Restriction prevents deletion of critical reference data (room_types → rooms).
-

Stored Procedure: Atomic Booking

- `sp_create_booking` handles booking creation **atomically**:
 1. Locks the room row to prevent concurrent bookings.
 2. Checks for overlapping bookings.
 3. Inserts booking and payment records.
 4. Updates room status to `OCCUPIED`.
 5. Logs the transaction.
 - Returns **success status** and **descriptive message**.
 - Uses **exception handling** and **ROLLBACK** to maintain integrity on errors.
-

Summary

This database is designed for **robust hotel booking operations**. By combining:

- Foreign key constraints for integrity
- Transactions for atomic operations
- Row-level locking to prevent conflicts
- JSON fields for flexible data
- Audit logs for traceability

...it ensures **high reliability, concurrency safety, and maintainability** in a hotel booking system.

Table	Related To	Relationship
users	bookings	1 → ∞
users	transaction_log	1 → ∞ (nullable FK)
hotels	rooms	1 → ∞
room_types	rooms	1 → ∞
rooms	bookings	1 → ∞
bookings	payments	1 → 1
bookings	transaction_log	1 → ∞ (nullable FK)

